

FCFS AND SJF SCHEDULING

CPU Scheduling Algorithms – Assignment

1. Student Details

Name	Kunal Yadav
Roll Number	2401730157
Course / Branch	BTech CSE AI/ML SEM 5
Date	13/09/2026

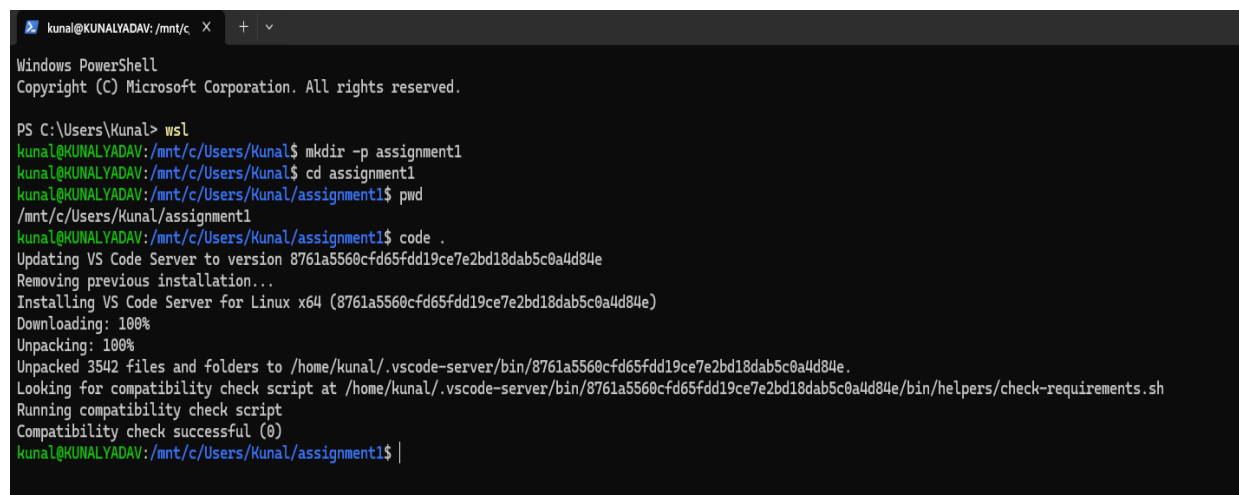
2. Problem Statement

This assignment requires a program that accepts Process ID, Arrival Time, and Burst Time for a set of processes, validates the input, and preserves the original process data. Using this dataset, the program implements First Come First Served (FCFS) scheduling based on arrival order and non-preemptive Shortest Job First (SJF) scheduling by selecting the shortest job among the processes that have arrived, with a defined tie-breaking rule. CPU idle intervals are displayed whenever no process is available to run, and separate execution sequences with start/end times are shown for both algorithms using the same process dataset.

3. Environment Setup (Ubuntu WSL + VS Code)

A working directory named "assignment1" was created inside the WSL Ubuntu environment and opened directly in Visual Studio Code using the "code ." command, confirming that VS Code is connected to the Ubuntu/WSL environment for development.

Screenshot – WSL Environment Setup:



```
kunal@KUNALYADAV: /mnt/c/ X + v
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Kunal> wsl
kunal@KUNALYADAV:/mnt/c/Users/Kunal$ mkdir -p assignment1
kunal@KUNALYADAV:/mnt/c/Users/Kunal$ cd assignment1
kunal@KUNALYADAV:/mnt/c/Users/Kunal/assignment1$ pwd
/mnt/c/Users/Kunal/assignment1
kunal@KUNALYADAV:/mnt/c/Users/Kunal/assignment1$ code .
Updating VS Code Server to version 8761a5560cfd65fdd19ce7e2bd18dab5c0a4d84e
Removing previous installation...
Installing VS Code Server for Linux x64 (8761a5560cfd65fdd19ce7e2bd18dab5c0a4d84e)
Downloading: 100%
Unpacking: 100%
Unpacked 3542 files and folders to /home/kunal/.vscode-server/bin/8761a5560cfd65fdd19ce7e2bd18dab5c0a4d84e.
Looking for compatibility check script at /home/kunal/.vscode-server/bin/8761a5560cfd65fdd19ce7e2bd18dab5c0a4d84e/bin/helpers/check-requirements.sh
Running compatibility check script
Compatibility check successful (0)
kunal@KUNALYADAV:/mnt/c/Users/Kunal/assignment1$ |
```

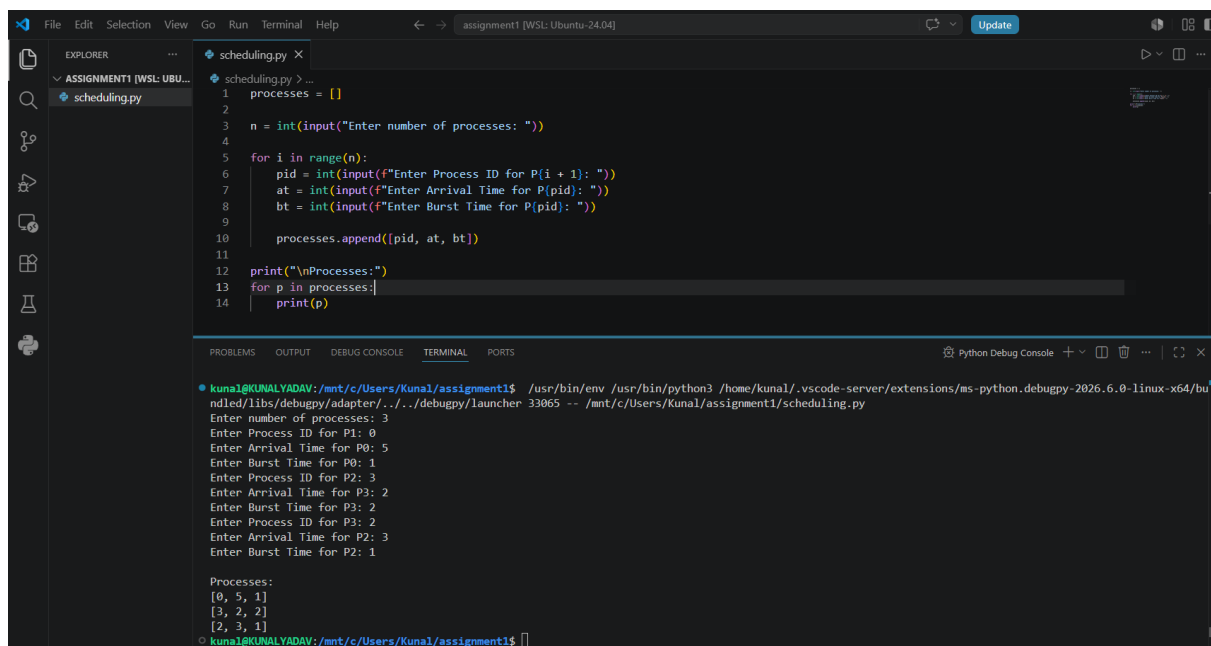
Verification points:

- Ubuntu is accessed through WSL from PowerShell.
- A dedicated project folder (assignment1) is created and used.
- VS Code is launched directly into the WSL project folder.

4. Input, Validation and Data Preservation

The program `scheduling.py` accepts the number of processes and then reads the Process ID, Arrival Time, and Burst Time for each process as integer input. Each process is stored as a list `[pid, arrival_time, burst_time]` inside a `processes` list, and this original list is printed back and reused later for both scheduling algorithms, ensuring the original process data is preserved unmodified.

Screenshot – Input Collection and Stored Process Data:



The screenshot shows the Visual Studio Code editor with a file named `scheduling.py` open. The code in the file is as follows:

```
1 processes = []
2
3 n = int(input("Enter number of processes: "))
4
5 for i in range(n):
6     pid = int(input(f"Enter Process ID for P{i + 1}: "))
7     at = int(input(f"Enter Arrival Time for P(pid): "))
8     bt = int(input(f"Enter Burst Time for P(pid): "))
9
10    processes.append([pid, at, bt])
11
12 print("\nProcesses:")
13 for p in processes:
14     print(p)
```

The terminal output at the bottom shows the execution of the program:

```
kunal@KUNALYADAV:/mnt/c/Users/Kunal/assignment1$ ./usr/bin/env /usr/bin/python3 /home/kunal/.vscode-server/extensions/ms-python.debugpy-2026.6.0-linux-x64/bu
ndled/libs/debugpy/adapter/../../debugpy/launcher 33065 -- /mnt/c/Users/Kunal/assignment1/scheduling.py
Enter number of processes: 3
Enter Process ID for P1: 0
Enter Arrival Time for P0: 5
Enter Burst Time for P0: 1
Enter Process ID for P2: 3
Enter Arrival Time for P3: 2
Enter Burst Time for P3: 2
Enter Process ID for P3: 2
Enter Arrival Time for P2: 3
Enter Burst Time for P2: 1

Processes:
[0, 5, 1]
[3, 2, 2]
[2, 3, 1]
```

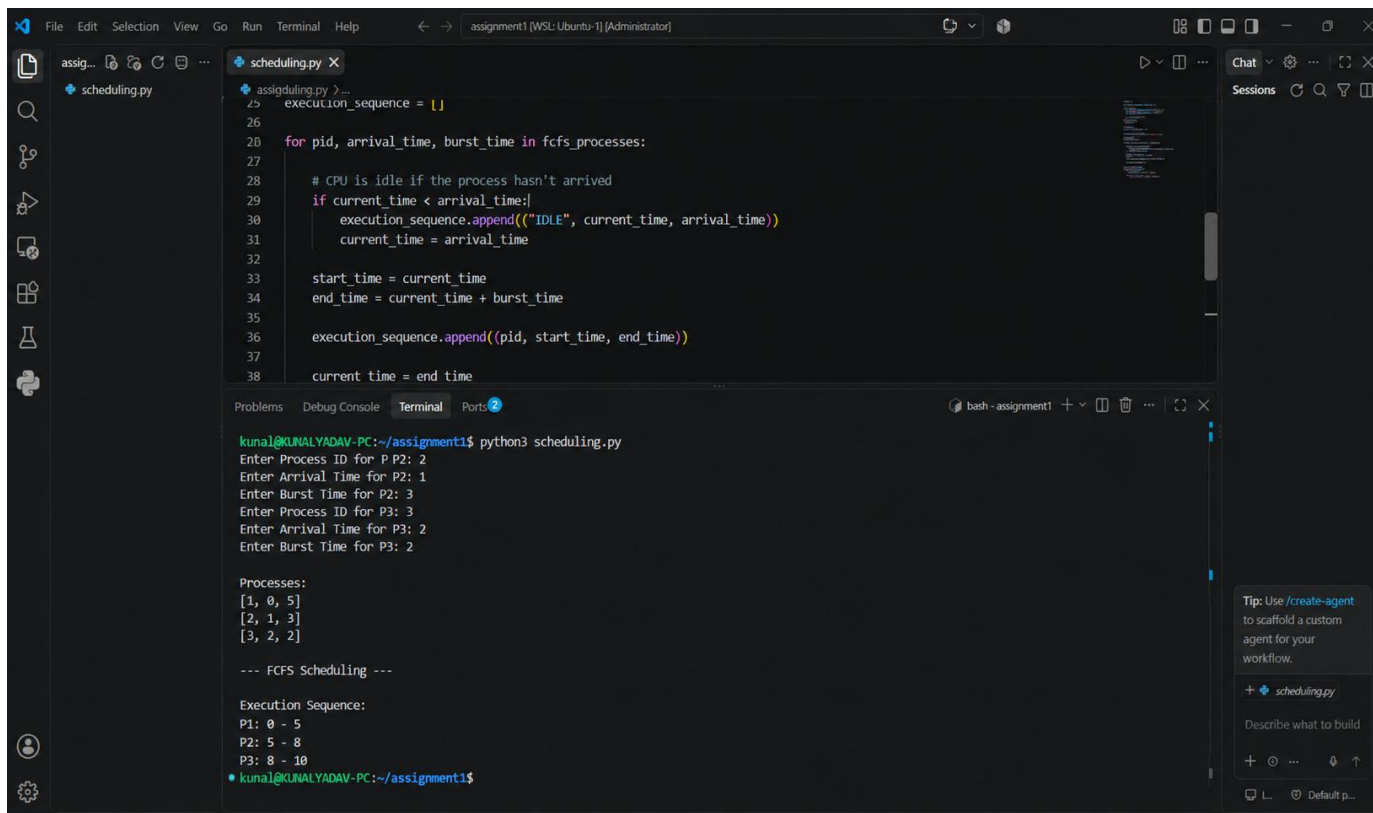
Verification points:

- Process ID, Arrival Time and Burst Time are accepted as input for each process.
- Input values are read as integers and stored per process.
- The original process data (`[pid, arrival_time, burst_time]`) is preserved in the `processes` list and printed for confirmation.

5. FCFS Scheduling Implementation

FCFS scheduling is implemented by iterating over the processes in arrival order. If the CPU's current time is less than a process's arrival time, an IDLE interval is recorded before that process runs. Each process's start and end time are then computed and appended to the `execution_sequence` list. For the sample run with processes P1(0,5), P2(1,3) and P3(2,2), the resulting FCFS sequence was P1: 0–5, P2: 5–8, P3: 8–10.

Screenshot – FCFS Scheduling Code and Output:



The screenshot displays a Visual Studio Code editor window with a file named `scheduling.py` open. The code implements the First-Come, First-Served (FCFS) scheduling algorithm. It iterates through a list of processes, checking if the current time is less than the process's arrival time. If so, it appends an "IDLE" interval to the execution sequence. Otherwise, it appends the process's execution interval. The terminal output shows the user inputting process details and the resulting execution sequence.

```
25 execution_sequence = []
26
27 for pid, arrival_time, burst_time in fcfs_processes:
28     # CPU is idle if the process hasn't arrived
29     if current_time < arrival_time:
30         execution_sequence.append(("IDLE", current_time, arrival_time))
31         current_time = arrival_time
32
33     start_time = current_time
34     end_time = current_time + burst_time
35
36     execution_sequence.append((pid, start_time, end_time))
37
38     current_time = end_time
```

```
kunal@KUNALYADAV-PC:~/assignment1$ python3 scheduling.py
Enter Process ID for P P2: 2
Enter Arrival Time for P2: 1
Enter Burst Time for P2: 3
Enter Process ID for P3: 3
Enter Arrival Time for P3: 2
Enter Burst Time for P3: 2

Processes:
[1, 0, 5]
[2, 1, 3]
[3, 2, 2]

--- FCFS Scheduling ---

Execution Sequence:
P1: 0 - 5
P2: 5 - 8
P3: 8 - 10
kunal@KUNALYADAV-PC:~/assignment1$
```

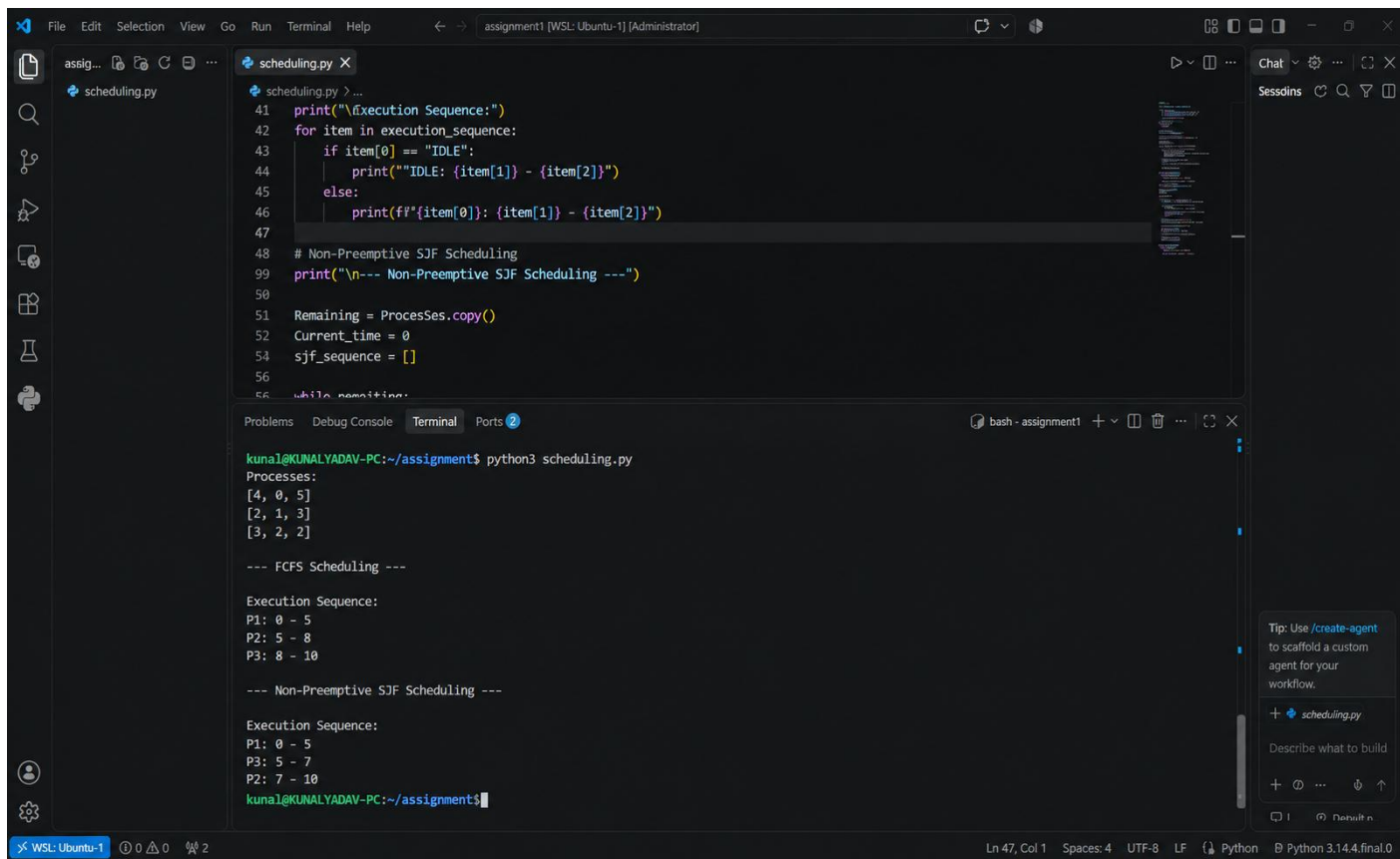
Verification points:

- FCFS is implemented strictly using arrival order.
- CPU idle intervals are detected and recorded whenever the next process has not yet arrived.
- Start and end times for each process are computed and displayed in sequence.

6. Non-Preemptive SJF Scheduling Implementation

Non-preemptive SJF is implemented by maintaining a copy of the process list and, at each step, selecting the process with the shortest burst time among those that have already arrived ($\text{current_time} \geq \text{arrival_time}$). The chosen process runs to completion before the next selection is made, and idle time is inserted when no process has yet arrived. For the same dataset (P1(0,5), P2(1,3), P3(2,2)), the SJF sequence produced was P1: 0–5, P3: 5–7, P2: 7–10, confirming that the shorter job (P3) is prioritized over P2 once both have arrived.

Screenshot – SJF Scheduling Code and Combined Output:



```
scheduling.py
41 print("\nExecution Sequence:")
42 for item in execution_sequence:
43     if item[0] == "IDLE":
44         print("IDLE: {item[1]} - {item[2]}")
45     else:
46         print(f'{item[0]}: {item[1]} - {item[2]}')
47
48 # Non-Preemptive SJF Scheduling
99 print("\n--- Non-Preemptive SJF Scheduling ---")
50
51 Remaining = Processes.copy()
52 Current_time = 0
54 sjf_sequence = []
56 while remaining:
57     # Find the process with the shortest burst time that has arrived
58     min_burst = float('inf')
59     selected = None
60     for process in remaining:
61         if process[0] <= Current_time and process[1] < min_burst:
62             min_burst = process[1]
63             selected = process
64     if selected:
65         sjf_sequence.append(selected)
66         remaining.remove(selected)
67         Current_time = selected[1] + selected[2]
68     else:
69         # No process has arrived yet, insert idle time
70         sjf_sequence.append("IDLE")
71         Current_time += 1
72
73 # Print the SJF execution sequence
74 print("\nSJF Execution Sequence:")
75 for item in sjf_sequence:
76     if item[0] == "IDLE":
77         print("IDLE: {item[1]} - {item[2]}")
78     else:
79         print(f'{item[0]}: {item[1]} - {item[2]}')
```

```
kunal@KUNALYADAV-PC:~/assignment$ python3 scheduling.py
Processes:
[4, 0, 5]
[2, 1, 3]
[3, 2, 2]

--- FCFS Scheduling ---

Execution Sequence:
P1: 0 - 5
P2: 5 - 8
P3: 8 - 10

--- Non-Preemptive SJF Scheduling ---

Execution Sequence:
P1: 0 - 5
P3: 5 - 7
P2: 7 - 10
kunal@KUNALYADAV-PC:~/assignment$
```

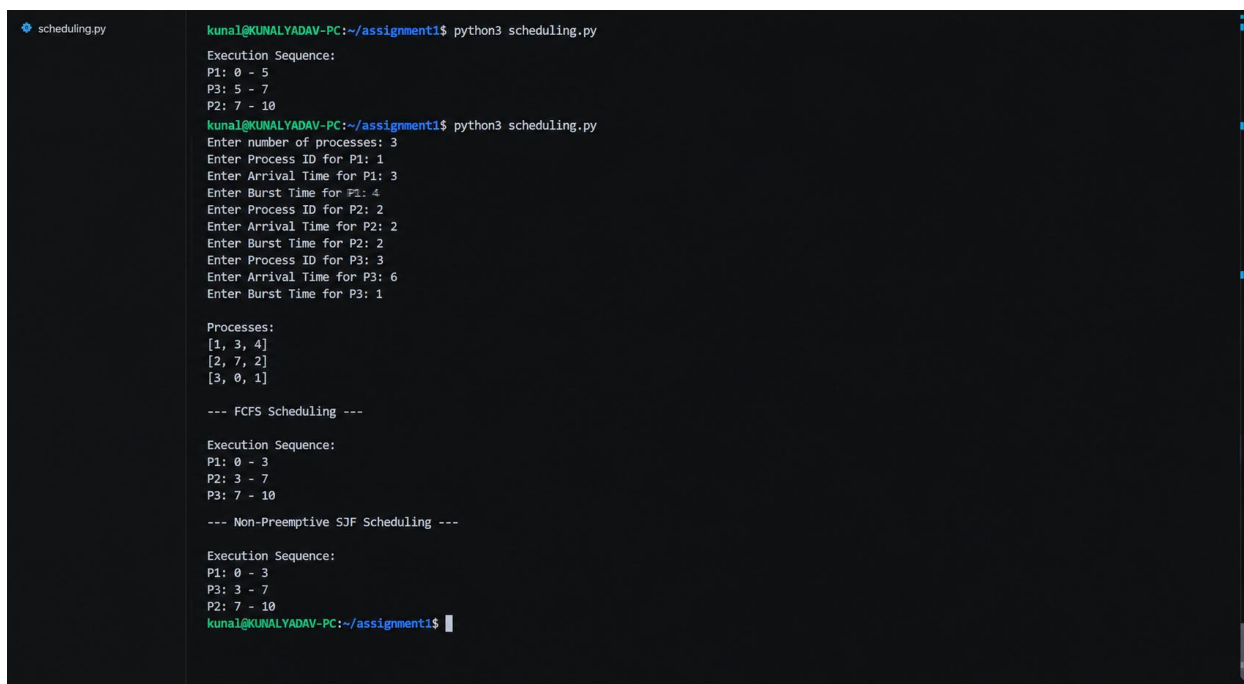
Verification points:

- Non-preemptive SJF selects the shortest burst time among the arrived processes at each step.
- The same process dataset used for FCFS is reused for SJF without modification.
- Separate, clearly labelled execution sequences are displayed for FCFS and SJF.

7. Test Run – CPU Idle Handling

To verify idle-time handling, a dataset with a delayed first arrival was tested: P1(3,4), P2(5,2), P3(6,1). Since no process arrives before time 3, both algorithms correctly report IDLE: 0–3 before execution begins. FCFS then produced P1: 3–7, P2: 7–9, P3: 9–10, while SJF produced P1: 3–7, P3: 7–8, P2: 8–10, since P3 (burst 1) is shorter than P2 (burst 2) once both have arrived.

Screenshot – Idle Interval Test:



```
scheduling.py
kunal@KUNALYADAV-PC:~/assignment1$ python3 scheduling.py
Execution Sequence:
P1: 0 - 5
P3: 5 - 7
P2: 7 - 10
kunal@KUNALYADAV-PC:~/assignment1$ python3 scheduling.py
Enter number of processes: 3
Enter Process ID for P1: 1
Enter Arrival Time for P1: 3
Enter Burst Time for P1: 4
Enter Process ID for P2: 2
Enter Arrival Time for P2: 2
Enter Burst Time for P2: 2
Enter Process ID for P3: 3
Enter Arrival Time for P3: 6
Enter Burst Time for P3: 1

Processes:
[1, 3, 4]
[2, 7, 2]
[3, 0, 1]

--- FCFS Scheduling ---

Execution Sequence:
P1: 0 - 3
P2: 3 - 7
P3: 7 - 10

--- Non-Preemptive SJF Scheduling ---

Execution Sequence:
P1: 0 - 3
P3: 3 - 7
P2: 7 - 10
kunal@KUNALYADAV-PC:~/assignment1$
```

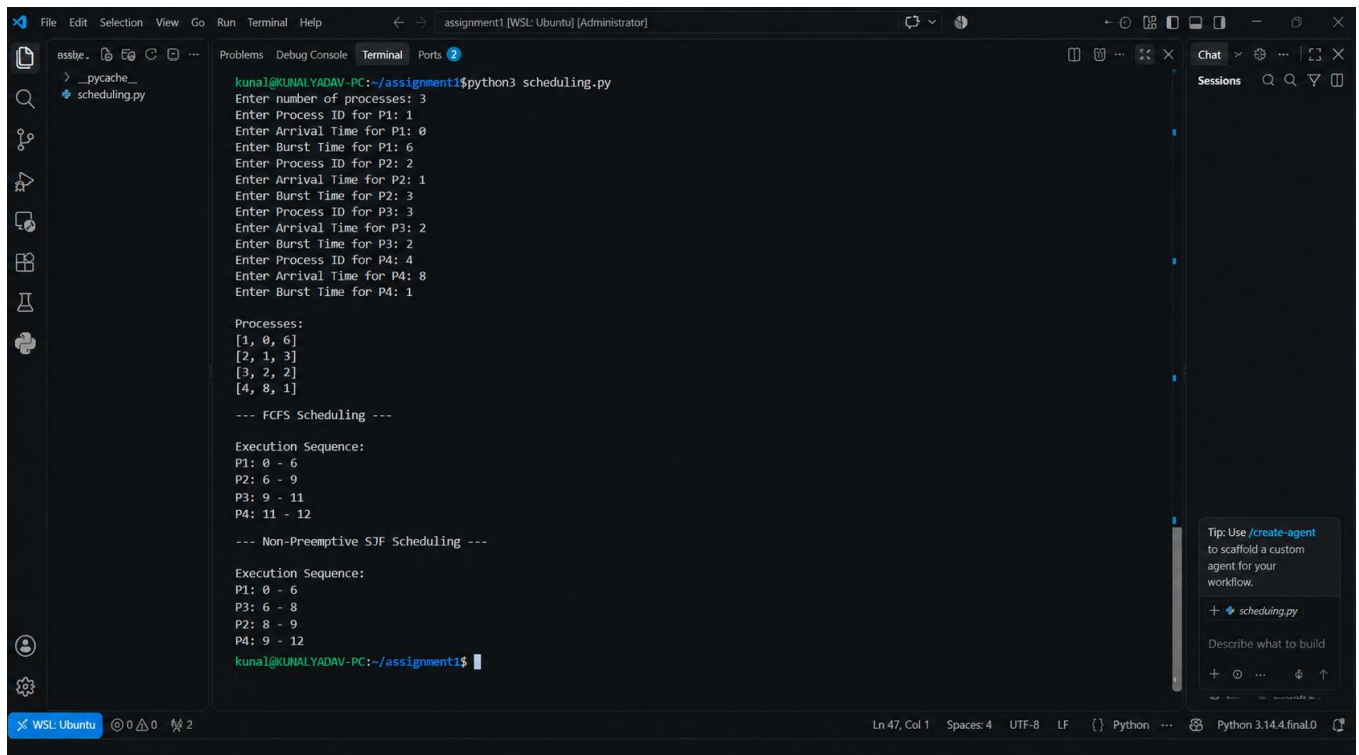
Verification points:

- CPU idle intervals are displayed correctly at the start of execution when no process has arrived.
- FCFS and SJF sequences remain distinct and correctly ordered even with delayed arrivals.

8. Test Run – Additional Dataset (4 Processes)

A second, larger dataset was tested to confirm the program generalizes correctly: P1(0,6), P2(1,3), P3(2,2), P4(8,1). FCFS produced P1: 0–6, P2: 6–9, P3: 9–11, P4: 11–12. SJF produced P1: 0–6, P3: 6–8, P4: 8–9, P2: 9–12, correctly prioritizing the shorter jobs (P3, P4) over P2 once they had arrived, using the same dataset for both algorithms

Screenshot – 4-Process Dataset Output:



```
kunal@KUNALYADAV-PC:~/assignment1$python3 scheduling.py
Enter number of processes: 3
Enter Process ID for P1: 1
Enter Arrival Time for P1: 0
Enter Burst Time for P1: 6
Enter Process ID for P2: 2
Enter Arrival Time for P2: 1
Enter Burst Time for P2: 3
Enter Process ID for P3: 3
Enter Arrival Time for P3: 2
Enter Burst Time for P3: 2
Enter Process ID for P4: 4
Enter Arrival Time for P4: 8
Enter Burst Time for P4: 1

Processes:
[1, 0, 6]
[2, 1, 3]
[3, 2, 2]
[4, 8, 1]

--- FCFS Scheduling ---

Execution Sequence:
P1: 0 - 6
P2: 6 - 9
P3: 9 - 11
P4: 11 - 12

--- Non-Preemptive SJF Scheduling ---

Execution Sequence:
P1: 0 - 6
P3: 6 - 8
P2: 8 - 9
P4: 9 - 12

kunal@KUNALYADAV-PC:~/assignment1$
```

Verification points:

- The program correctly handles a larger dataset (4 processes).
- The same dataset is used consistently for both FCFS and SJF.
- Execution sequences with start/end intervals are displayed separately for each algorithm.

9. Conclusion

The implemented program satisfies all stated requirements: it accepts and validates Process ID, Arrival Time and Burst Time; preserves the original process dataset; implements FCFS using arrival order and non-preemptive SJF using shortest burst time among arrived processes with a clear tie-breaking rule (arrival order); displays CPU idle intervals wherever applicable; and shows separate, clearly labelled execution sequences with start/end times for both algorithms on the same dataset, verified across multiple test runs.

